

TRABAJO DE DOCUMENTACIÓN

Sistemas Informáticos

1º DAM / 1º DAW



Instalación, despliegue y funcionamiento de aplicaciones Java en entornos reales

Realizado por: Rubén, Juan Pablo, Hector y Leo

Índice:

- **1.** Entorno de ejecución de aplicaciones Java.
- **2.** Creación de instaladores para aplicaciones Java.
- **3.** Servicios, APIs y comunicación en red.
- **4.** Contenedores con Docker.
- **5.** Aplicaciones frontend con Angular y React.

1. Entorno de ejecución de aplicaciones Java

- **Investiga qué necesita un equipo para ejecutar aplicaciones Java. Explica qué es la JVM y por qué permite ejecutar programas Java en distintos sistemas operativos.**

Para que una aplicación pueda ejecutar programas de java necesita el Entorno de Ejecución de Java (JRE - Java Runtime Environment), que incluye la Máquina Virtual de Java (JVM), o bien el Kit de Desarrollo de Java (JDK - Java Development Kit) si además de ejecutarlas se van a programar

La JVM interpreta y ejecuta el bytecode (código intermedio .class) en código máquina nativo del sistema operativo sobre el que corre, también gestiona la memoria y verifica el código asegurándose que sea seguro antes de ejecutarlo.

- **Diferencia entre JDK y JRE... Indica cuál se necesita para desarrollar y cuál para ejecutar aplicaciones**

El **JDK** (Java Development Kit) es el kit de herramientas necesario para desarrollar (programar y compilar) aplicaciones Java, mientras que el **JRE** (Java Runtime Environment) es el entorno necesario únicamente para ejecutar aplicaciones ya creadas

- **Describe los pasos básicos para instalar Java en Windows o Linux y cómo comprobar la instalación desde la terminal con comandos como java -version**

En Windows, la forma más común es utilizar el instalador ejecutable oficial.

1. **Descarga el JDK:** Ve al sitio web de Oracle o usa una distribución abierta como Adoptium (Eclipse Temurin). Descarga el instalador .msi o .exe para x64.
2. **Ejecuta el instalador:** Sigue los pasos del asistente
3. Por defecto, se instalará en C:\Program Files\Java\jdk-xx.
4. **Configura las Variables de Entorno:**

Busca "Editar las variables de entorno del sistema" en el menú Inicio.

En Variables del sistema, haz clic en Nueva y crea JAVA_HOME con la ruta de tu carpeta JDK.

Busca la variable Path, editala y añade %JAVA_HOME%\bin.

- **Explica variables JAVA_HOME y PATH... para qué sirven y qué problemas pueden aparecer si no están configuradas correctamente**

1. JAVA_HOME:

- **Para qué sirve:** Indica al sistema operativo y a las herramientas de desarrollo (Maven, Tomcat, Eclipse, etc.) dónde está instalada la carpeta raíz del JDK (Java Development Kit).
- **Uso:** Generalmente, se configura para apuntar a la ruta principal, por ejemplo: C:\Program Files\Java\jdk-17.

2. PATH:

- **Para qué sirve:** Es una variable del sistema que contiene una lista de directorios. Le dice al sistema operativo dónde buscar los archivos ejecutables (java.exe, javac.exe) cuando escribes un comando en la consola.
- **Uso:** Se debe incluir la carpeta bin dentro del JDK, por ejemplo: %JAVA_HOME%\bin (o la ruta directa).

2. Creación de instaladores

- **Define qué es un instalador y qué ventajas tiene frente a entregar simplemente un archivo comprimido o un .jar.**

Un instalador es un software diseñado específicamente para guiar al usuario en el proceso de transferir, configurar y preparar una aplicación para su uso en un sistema operativo determinado a diferencia de un simple archivo comprimido (ZIP o RAR) o un archivo Java ejecutable (.jar), un instalador realiza una serie de tareas automatizadas que garantizan que el programa funcione correctamente en el entorno del usuario, a su vez un archivo o comprimido si el usuario no tiene Java instalado o tiene una versión incompatible, el archivo simplemente no abrirá o lanzará un error críptico.

- Investiga herramientas que permitan generar instaladores o ejecutables para aplicaciones Java, por ejemplo Inno Setup, install4j, jpackage o Launch4j.

1. jpackage (Recomendada y Moderna):

Integrada en el JDK desde la versión 16, es la solución estándar moderna de Oracle para empaquetar aplicaciones Java como instaladores nativos.

- Tipo: Herramienta de línea de comandos incluida en el JDK.
- Ventajas: Permite crear una imagen de tiempo de ejecución pequeña y personalizada (usando jlink) para agruparla con la aplicación.
- Ideal para: Aplicaciones modernas, JavaFX, Swing y herramientas CLI que buscan una integración nativa total.

2. Launch4j:

Es un "wrapper" de código abierto que convierte archivos .jar en ejecutables .exe nativos de Windows.

- Tipo: Interfaz gráfica (GUI) y CLI para envolver JARs.
- Funciones: Permite definir un JRE mínimo, establecer íconos, splash screens y gestionar parámetros de la JVM.
- Limitación: No es un instalador completo, solo genera el ejecutable. Para crear un instalador (ej. setup.exe), se suele combinar con *Inno Setup*.
- Ideal para: Crear un ejecutable portable rápido y sencillo para Windows.

3. Inno Setup:

Es una herramienta clásica, gratuita y muy potente para crear instaladores profesionales en Windows.

- Tipo: Generador de instaladores basado en scripts (con asistente GUI).
- Uso: Toma la aplicación (o el .exe generado por Launch4j) y crea un instalador completo con desinstalador, accesos directos, etc..
- Ideal para: Situaciones donde necesitas un control total sobre el proceso de instalación en Windows, incluyendo licencias y dependencias extra.

4. install4j:

Es una de las herramientas más completas y profesionales.

- Tipo: Creador de instaladores GUI avanzado.
- Ventajas: Facilidad de uso superior, excelente soporte multiplataforma, detección/empaquetado automático de JRE, y capacidades de actualización automática.
- Ideal para: Empresas o proyectos comerciales que requieren un instalador de alta calidad con soporte técnico y funciones avanzadas sin escribir scripts.

• **Explica qué elementos puede incluir un instalador: archivos de la aplicación, accesos directos, configuración inicial, dependencias, desinstalador y rutas de instalación.**

Archivos de la aplicación: Son el núcleo del instalador. Incluye el ejecutable principal, librerías de enlace dinámico, archivos de configuración, imágenes, sonidos y otros recursos necesarios.

Dependencias: El instalador a menudo verifica e instala componentes adicionales que el programa necesita para funcionar, como bibliotecas de tiempo de ejecución, frameworks o controladores.

Rutas de instalación: Define dónde se copiarán los archivos. Por defecto, en Windows suele ser C:\Program Files o C:\Program Files (x86), pero el instalador permite al usuario seleccionar una ubicación personalizada.

Accesos directos: Crea iconos en el escritorio, el menú Inicio, la barra de tareas o en la carpeta de inicio automático para facilitar el acceso rápido del usuario al programa.

Configuración inicial: Puede incluir la modificación del Registro de Windows, el establecimiento de variables de entorno, la creación de asociaciones de archivo o una primera configuración del usuario.

Desinstalador: Es una herramienta vital que crea el propio instalador para eliminar de forma ordenada y completa todos los archivos, registros y accesos directos creados, asegurando que no queden archivos basura en el sistema.

3. Servicios, APIs y comunicación en red

- **Explica qué es una API y qué papel tiene dentro del modelo cliente-servidor. El objetivo es entender la comunicación entre aplicaciones, no programar una API completa.**

Una API es un conjunto de reglas, protocolos y definiciones que permiten a aplicaciones que se comuniquen entre sí.

Actúa como un puente o intermediario que facilita el intercambio de datos, características y funcionalidades sin necesidad de conocer cómo está implementado el otro sistema.

- **Investiga qué es Spring Boot y por qué se utiliza para crear servicios o APIs en Java.**

Spring Boot es un framework de código abierto basado en Java, parte del ecosistema Spring, diseñado para simplificar drásticamente la creación, configuración y despliegue de aplicaciones independientes y listas para producción.

- **Describe conceptos básicos de red relacionados con una API: dirección IP, puerto, protocolo HTTP/HTTPS, peticiones GET y POST, y respuesta del servidor.**

Dirección IP: Es una etiqueta numérica única que identifica a un dispositivo en una red.

Puerto: Es un número que identifica una aplicación o servicio específico dentro de un servidor.

Protocolo HTTP/HTTPS: Conjunto de reglas que definen cómo se formatean y transmiten los mensajes entre un cliente y un servidor.

Peticiones GET y POST: Son métodos, que indican la acción a realizar.

4. Contenedores con Docker

- **Explica qué es Docker y qué problema intenta resolver: ejecutar aplicaciones en entornos aislados y reproducibles.**

Docker permite ejecutar aplicaciones en entornos aislados que tienen todo lo necesario para funcionar. Busca resolver problemas de compatibilidad entre distintos entornos, garantizando que el programa se ejecute correctamente en cualquier sistema. Además, los contenedores son más ligeros y rápidos que las máquinas virtuales.

- **Diferencia entre máquina virtual y contenedor. Indica ventajas e inconvenientes de cada enfoque de forma sencilla.**

La diferencia es que una máquina virtual incluye un sistema operativo completo, esto hace que sea más pesada y lenta. Por otro lado, un contenedor comparte el sistema operativo de tu ordenador y solamente aísla la aplicación, por lo que es más rápida y eficiente, aunque con un aislamiento menor. En conclusión, las máquinas virtuales priorizan la seguridad y el aislamiento, mientras que los contenedores se centran en la rapidez y la eficiencia.

- **Define los conceptos básicos: imagen, contenedor, Dockerfile, volumen y puerto expuesto.**

Una imagen es una plantilla con todo lo que se necesita para ejecutar una aplicación. Un contenedor es esa aplicación ya funcionando en el ordenador de forma aislada, el Dockerfile es el archivo donde se define cómo construir la imagen, un volumen permite guardar datos de forma permanente fuera del contenedor, y un puerto permite la comunicación entre el contenedor y el exterior.

- **Relaciona Docker con Java: cómo podría empaquetar una aplicación Spring Boot en una imagen y ejecutarse en cualquier equipo con Docker instalado.**

Docker permite ejecutar aplicaciones Java en cualquier equipo sin depender de la configuración del sistema. La aplicación se empaqueta en una imagen mediante un Dockerfile, donde se indica la versión de Java necesaria y el archivo.jar que tiene la aplicación. Esa imagen y luego se ejecuta como un contenedor. Así, la aplicación incluye todo lo que necesita para funcionar y puede ejecutarse igual en cualquier ordenador con Docker.

5. Aplicaciones frontend con Angular y React

- **Explica qué es una aplicación frontend y su papel en cliente-servidor.**

Una aplicación frontend es la parte visual con la que interactúa el usuario (HTML, CSS, JS). En el modelo cliente-servidor, el frontend (cliente) hace peticiones al backend (servidor) y muestra los datos que recibe.

- **Investiga Angular y React.**

Angular es un framework completo de frontend hecho por Google, usa TypeScript y tiene todo integrado (routing, formularios, etc.).

React es una librería de Facebook para construir interfaces mediante componentes, más flexible pero necesitas añadir otras librerías para tener todo.

- **Describe ejecución: Node.js, npm, servidor dev.**

Node.js permite ejecutar JavaScript fuera del navegador. npm es el gestor de paquetes para instalar dependencias. El servidor de desarrollo (dev server) sirve la app en local con recarga automática mientras programas.

- **Explica build y archivos generados.**

El build es el proceso de preparar la app para producción: se optimiza, minimiza y empaqueta. Genera archivos estáticos (HTML, CSS, JS) listos para desplegar.

- **Describe despliegue en servidor web.**

Consiste en subir los archivos del build a un servidor web (como Apache o Nginx) para que los usuarios accedan desde internet.

- **Relación con backend Java y APIs.**

El frontend se comunica con el backend Java (por ejemplo Spring Boot) mediante APIs REST. Hace peticiones HTTP (GET, POST, etc.) y recibe datos en JSON para mostrarlos.